

TP - ANDROID - Lab 4 : Affichage d'une liste de livres avec RecyclerView

1. Objectifs pédagogiques

À la fin de ce TP, l'étudiant sera capable de :

- Créer une classe métier Java `Livre`
- Manipuler une `ArrayList` d'objets
- Comprendre le rôle d'un `RecyclerView`
- Créer un `RecyclerView.Adapter` personnalisé en Java
- Créer un `ViewHolder` statique interne à un adapter
- Concevoir un layout XML pour représenter un élément de liste
- Relier les données Java à l'interface graphique
- Afficher une liste de livres dans l'application de bibliothèque municipale

2. Prérequis techniques

Avant de commencer, vérifier que le projet contient les dépendances nécessaires dans le fichier `build.gradle` du module `app`.

Dépendances à ajouter

```
1 dependencies {
2     implementation(libs.appcompat)
3     implementation(libs.material)
4     implementation(libs.activity)
5     implementation(libs.constraintlayout)
6     // Dépendances à ajouter
7     implementation(libs.recyclerview)
8     implementation(libs.cardview)
9
10    testImplementation(libs.junit)
11    androidTestImplementation(libs.ext.junit)
12    androidTestImplementation(libs.espresso.core)
13 }
```

Après modification :

- cliquer sur **Sync Now**
- attendre la fin de la synchronisation Gradle

i Si `libs.recyclerview` et `libs.cardview` n'existent pas.

Vérifier le fichier :

```
1 | gradle/libs.versions.toml
```

Ajouter :

```
1 [versions]
2 recyclerview = "1.3.2"
3 cardview = "1.0.0"
4
5 [libraries]
6 recyclerview = { module = "androidx.recyclerview:recyclerview", version.ref =
  "recyclerview" }
7 cardview = { module = "androidx.cardview:cardview", version.ref = "cardview" }
```

Explication

- `libs.xxx` = alias défini dans `libs.versions.toml`
- Permet de **centraliser les versions**
- Évite de répéter les numéros de version dans tous les fichiers

3. Mise en situation (contexte métier)

Dans le Lab 3, vous avez créé l'écran d'accueil de l'application de la bibliothèque municipale.

Dans ce nouveau lab, vous allez afficher la **liste des livres disponibles dans la bibliothèque**.

L'objectif est de construire un écran capable d'afficher plusieurs livres sous forme de cartes, avec pour chaque livre :

- son titre
- son auteur
- son statut de disponibilité

Cet affichage sera réalisé avec **RecyclerView**, un composant Android très utilisé pour afficher des listes de manière performante.

4. Étapes de réalisation

Étape 1 : Préparer le nouvel écran principal

Dans ce lab, l'écran principal affichera directement la liste des livres.

Ouvrir :

```
1 res/layout/activity_main.xml
```

Remplacer le contenu par :

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <androidx.constraintlayout.widget.ConstraintLayout
3     xmlns:android="http://schemas.android.com/apk/res/android"
4     xmlns:app="http://schemas.android.com/apk/res-auto"
5     xmlns:tools="http://schemas.android.com/tools"
```

```

6     android:layout_width="match_parent"
7     android:layout_height="match_parent"
8     tools:context=".MainActivity">
9
10    <!-- Titre de l'écran -->
11    <TextView
12        android:id="@+id/tvTitreEcran"
13        android:layout_width="wrap_content"
14        android:layout_height="wrap_content"
15        android:text="Liste des livres"
16        android:textSize="22sp"
17        android:textStyle="bold"
18        android:layout_marginTop="16dp"
19        android:layout_marginStart="16dp"
20        app:layout_constraintTop_toTopOf="parent"
21        app:layout_constraintStart_toStartOf="parent" />
22
23    <!-- RecyclerView qui affichera les livres -->
24    <androidx.recyclerview.widget.RecyclerView
25        android:id="@+id/recyclerViewLivres"
26        android:layout_width="0dp"
27        android:layout_height="0dp"
28        android:layout_marginTop="16dp"
29        android:layout_marginStart="12dp"
30        android:layout_marginEnd="12dp"
31        android:layout_marginBottom="12dp"
32        app:layout_constraintTop_toBottomOf="@id/tvTitreEcran"
33        app:layout_constraintBottom_toBottomOf="parent"
34        app:layout_constraintStart_toStartOf="parent"
35        app:layout_constraintEnd_toEndOf="parent" />
36
37 </androidx.constraintlayout.widget.ConstraintLayout>

```

Explication

- Le `TextView` affiche le titre de l'écran
- Le `RecyclerView` servira de conteneur pour afficher la liste des livres
- Contrairement à un `LinearLayout`, le `RecyclerView` est conçu pour gérer efficacement de longues listes

Étape 2 : Créer la classe métier Livre.java

Créer une nouvelle classe Java nommée :

```
1 | Livre.java
```

Contenu :

```

1 public class Livre {
2
3     // Identifiant unique du livre
4     private int id;
5
6     // Titre du livre
7     private String titre;
8
9     // Nom de l'auteur
10    private String auteur;
11
12    // Code ISBN du livre

```

```
13     private String isbn;
14
15     // Indique si le livre est disponible ou non
16     private boolean disponible;
17
18     // Constructeur complet
19     public Livre(int id, String titre, String auteur, String isbn,
20 boolean disponible) {
21         this.id = id;
22         this.titre = titre;
23         this.auteur = auteur;
24         this.isbn = isbn;
25         this.disponible = disponible;
26     }
27
28     // Getter pour l'id
29     public int getId() {
30         return id;
31     }
32
33     // Setter pour l'id
34     public void setId(int id) {
35         this.id = id;
36     }
37
38     // Getter pour le titre
39     public String getTitre() {
40         return titre;
41     }
42
43     // Setter pour le titre
44     public void setTitre(String titre) {
45         this.titre = titre;
46     }
47
48     // Getter pour l'auteur
49     public String getAuteur() {
50         return auteur;
51     }
52
53     // Setter pour l'auteur
54     public void setAuteur(String auteur) {
55         this.auteur = auteur;
56     }
57
58     // Getter pour l'isbn
59     public String getIsbn() {
60         return isbn;
61     }
62
63     // Setter pour l'isbn
64     public void setIsbn(String isbn) {
65         this.isbn = isbn;
66     }
67
68     // Getter pour la disponibilité
69     public boolean isDisponible() {
70         return disponible;
71     }
72
73     // Setter pour la disponibilité
74     public void setDisponible(boolean disponible) {
75         this.disponible = disponible;
76     }
77 }
```

Explication

Cette classe représente un livre de la bibliothèque.

Chaque objet `Livre` contient les informations nécessaires pour être affiché dans la liste.

Étape 3 : Créer le layout d'un élément de liste

Créer un nouveau fichier dans :

```
1 | res/layout/item_livre.xml
```

Contenu :

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <androidx.cardview.widget.CardView
3     xmlns:android="http://schemas.android.com/apk/res/android"
4     xmlns:app="http://schemas.android.com/apk/res-auto"
5     android:layout_width="match_parent"
6     android:layout_height="wrap_content"
7     android:layout_margin="8dp"
8     app:cardCornerRadius="12dp"
9     app:cardElevation="4dp">
10
11     <LinearLayout
12         android:layout_width="match_parent"
13         android:layout_height="wrap_content"
14         android:orientation="vertical"
15         android:padding="16dp">
16
17         <!-- Titre du livre -->
18         <TextView
19             android:id="@+id/tvTitreLivre"
20             android:layout_width="wrap_content"
21             android:layout_height="wrap_content"
22             android:text="Titre du livre"
23             android:textSize="18sp"
24             android:textStyle="bold" />
25
26         <!-- Auteur du livre -->
27         <TextView
28             android:id="@+id/tvAuteurLivre"
29             android:layout_width="wrap_content"
30             android:layout_height="wrap_content"
31             android:layout_marginTop="6dp"
32             android:text="Auteur"
33             android:textSize="15sp" />
34
35         <!-- ISBN du livre -->
36         <TextView
37             android:id="@+id/tvIsbnLivre"
38             android:layout_width="wrap_content"
39             android:layout_height="wrap_content"
40             android:layout_marginTop="4dp"
41             android:text="ISBN : 0000000000"
42             android:textSize="13sp" />
43
44         <!-- Badge de disponibilité -->
45         <TextView
46             android:id="@+id/tvDisponibilite"
47             android:layout_width="wrap_content"
48             android:layout_height="wrap_content"
49             android:layout_marginTop="10dp"
```

```

50         android:paddingStart="10dp"
51         android:paddingTop="6dp"
52         android:paddingEnd="10dp"
53         android:paddingBottom="6dp"
54         android:text="Disponible"
55         android:textStyle="bold"
56         android:textColor="@android:color/white"
57         android:background="@android:color/holo_green_dark" />
58     </LinearLayout>
59 </androidx.cardview.widget.CardView>
60
61 </androidx.cardview.widget.CardView>

```

Explication

Ce fichier XML définit l'apparence d'un seul livre dans la liste.

Chaque élément sera affiché sous forme de carte grâce à `CardView`.

Étape 4 : Créer l'adapter personnalisé

Créer une nouvelle classe Java nommée :

```
1 LivreAdapter.java
```

Contenu :

```

1 import android.view.LayoutInflater;
2 import android.view.View;
3 import android.view.ViewGroup;
4 import android.graphics.Color;
5 import android.widget.TextView;
6
7 import androidx.annotation.NonNull;
8 import androidx.recyclerview.widget.RecyclerView;
9
10 import java.util.ArrayList;
11
12 public class LivreAdapter extends
13     RecyclerView.Adapter<LivreAdapter.LivreViewHolder> {
14
15     // Liste des livres à afficher
16     private ArrayList<Livre> listeLivres;
17
18     // Constructeur de l'adapter
19     public LivreAdapter(ArrayList<Livre> listeLivres) {
20         this.listeLivres = listeLivres;
21     }
22
23     @NonNull
24     @Override
25     public LivreViewHolder onCreateViewHolder(@NonNull ViewGroup
26     parent, int viewType) {
27         // Convertit le fichier XML item_livre.xml en objet View
28         View view = LayoutInflater.from(parent.getContext())
29             .inflate(R.layout.item_livre, parent, false);
30
31         return new LivreViewHolder(view);
32     }
33
34     @Override
35     public void onBindViewHolder(@NonNull LivreViewHolder holder, int
36     position) {

```

```

34 // Récupère le livre à la position courante
35 Livre livre = listLivres.get(position);
36
37 // Injecte les données du livre dans les vues
38 holder.tvTitreLivre.setText(livre.getTitre());
39 holder.tvAuteurLivre.setText("Auteur : " + livre.getAuteur());
40 holder.tvIsbnLivre.setText("ISBN : " + livre.getIsbn());
41
42 // Gestion du badge de disponibilité
43 if (livre.isDisponible()) {
44     holder.tvDisponibilite.setText("Disponible");
45     holder.tvDisponibilite.setBackgroundColor(Color.parseColor("#2E7D32"))
46     ;
47     } else {
48     holder.tvDisponibilite.setText("Indisponible");
49     holder.tvDisponibilite.setBackgroundColor(Color.parseColor("#C62828"))
50     ;
51     }
52
53 @Override
54 public int getItemCount() {
55     // Retourne le nombre total d'éléments à afficher
56     return listLivres.size();
57 }
58
59 // ViewHolder statique interne
60 public static class LivreViewHolder extends
61 RecyclerView.ViewHolder {
62
63     TextView tvTitreLivre;
64     TextView tvAuteurLivre;
65     TextView tvIsbnLivre;
66     TextView tvDisponibilite;
67
68     public LivreViewHolder(@NonNull View itemView) {
69         super(itemView);
70
71         // Liaison entre les composants XML et les variables Java
72         tvTitreLivre = itemView.findViewById(R.id.tvTitreLivre);
73         tvAuteurLivre = itemView.findViewById(R.id.tvAuteurLivre);
74         tvIsbnLivre = itemView.findViewById(R.id.tvIsbnLivre);
75         tvDisponibilite =
76         itemView.findViewById(R.id.tvDisponibilite);
77     }
78 }

```

Explication

L'adapter sert d'intermédiaire entre :

- les données Java (`ArrayList<Livre>`)
- l'affichage à l'écran (`RecyclerView`)

Le `ViewHolder` permet de stocker les références des composants d'un item pour éviter de les rechercher à chaque affichage.

Étape 5 : Préparer la liste statique de livres dans MainActivity

Ouvrir :

```
1 MainActivity.java
```

Remplacer le contenu par :

```
1 import android.os.Bundle;
2
3 import androidx.appcompat.app.AppCompatActivity;
4 import androidx.recyclerview.widget.LinearLayoutManager;
5 import androidx.recyclerview.widget.RecyclerView;
6
7 import java.util.ArrayList;
8
9 public class MainActivity extends AppCompatActivity {
10
11     // Composant d'affichage de liste
12     private RecyclerView recyclerViewLivres;
13
14     // Adapter personnalisé
15     private LivreAdapter livreAdapter;
16
17     // Liste des livres
18     private ArrayList<Livre> listeLivres;
19
20     @Override
21     protected void onCreate(Bundle savedInstanceState) {
22         super.onCreate(savedInstanceState);
23
24         // Associe le layout XML à l'activité
25         setContentView(R.layout.activity_main);
26
27         // Récupération du RecyclerView dans le layout
28         recyclerViewLivres = findViewById(R.id.recyclerViewLivres);
29
30         // Initialisation de la liste
31         listeLivres = new ArrayList<>();
32
33         // Ajout de 8 livres fictifs
34         listeLivres.add(new Livre(1, "Le Petit Prince", "Antoine de
35 Saint-Exupéry", "9780156013987", true));
36         listeLivres.add(new Livre(2, "L'Étranger", "Albert Camus",
37 "9782070360024", false));
38         listeLivres.add(new Livre(3, "Les Misérables", "Victor Hugo",
39 "9782253096344", true));
40         listeLivres.add(new Livre(4, "Une si longue lettre", "Mariama
41 Bâ", "9782841290529", true));
42         listeLivres.add(new Livre(5, "Le Vieux Nègre et la Médaille",
43 "Ferdinand Oyono", "9782264018304", false));
44         listeLivres.add(new Livre(6, "Madame Bovary", "Gustave
45 Flaubert", "9782070409228", true));
46         listeLivres.add(new Livre(7, "La Peste", "Albert Camus",
47 "9782070360420", false));
48         listeLivres.add(new Livre(8, "Sous l'orage", "Seydou Badian",
49 "9782708707691", true));
50
51         // Le RecyclerView affichera les éléments verticalement
52         recyclerViewLivres.setLayoutManager(new
53 LinearLayoutManager(this));
54
55         // Création de l'adapter avec la liste des livres
56         livreAdapter = new LivreAdapter(listeLivres);
57     }
58 }
```



```
49      // Liaison entre le RecyclerView et l'adapter
50      recyclerViewLivres.setAdapter(livreAdapter);
51  }
52 }
```

Explication

Dans cette étape :

- on crée une liste de 8 livres fictifs
- on configure le `RecyclerView` avec un `LinearLayoutManager`
- on crée l'adapter
- on branche l'adapter au `RecyclerView`

C'est cette liaison qui permet l'affichage de la liste.

Étape 6 : Tester l'application

Lancer l'application sur l'émulateur ou sur un téléphone.

Vérifier que :

- le titre "Liste des livres" s'affiche
- les 8 livres apparaissent sous forme de cartes
- chaque carte affiche le titre, l'auteur, l'ISBN et le statut
- les badges changent selon la disponibilité

5. Résumé des fichiers créés ou modifiés

Fichiers créés

- `Livre.java`
- `LivreAdapter.java`
- `res/layout/item_livre.xml`

Fichiers modifiés

- `MainActivity.java`
- `res/layout/activity_main.xml`
- `build.gradle` (Module: app)

6. Rendu attendu

L'application doit afficher un écran contenant :

- un titre en haut : **Liste des livres**

- une liste verticale de **8 livres**
- chaque livre affiché dans une **carte**
- dans chaque carte :
 - le titre
 - l'auteur
 - l'ISBN
 - un badge de disponibilité

Description de la capture attendue

La capture doit montrer :

- un écran propre avec plusieurs cartes empilées verticalement
- des titres de livres bien visibles
- les noms des auteurs
- un badge vert pour les livres disponibles
- un badge rouge pour les livres indisponibles

7. Bonus : explorer pour améliorer le design

Pour aller plus loin, vous pouvez :

- changer les couleurs des cartes
- ajouter une icône de livre dans chaque item
- améliorer le badge de disponibilité avec un fond arrondi
- ajouter des espacements plus harmonieux
- personnaliser la police et la taille du texte
- centrer ou mieux hiérarchiser les informations
- ajouter une bordure ou une ombre plus visible sur les cartes